Support Vector Machines
from Scratch

Line-by-line theoretical commentary on `svm_from_scratch.py`

Lecture 02 — Finance Course

This document is a companion to `svm_from_scratch.py`. Every code block is reproduced verbatim, followed by a *line-by-line walkthrough*, the math behind it, and the practical pitfalls. Slides from Igor Halperin's *Supervised, Unsupervised and Reinforcement Learning in Finance* (NYU Tandon, 2017) are cross-referenced where relevant. Blue DERIVATION boxes carry the full algebra.

The slides are organised in three parts:

- **Part I** — primal SVR / SVC, ε-insensitive loss, soft margin C.
- **Part II** — KKT conditions, the dual problem, the support-vector expansion, convex optimisation.
- **Part III** — the kernel trick: features $\Phi(x)$ replaced by a kernel $k(x, x')$.

1. Mathematical recap

1.1 Primal soft-margin SVC (Part I)

The primal problem minimises the squared norm of the weight vector (equivalently, maximises the margin width $2/\|w\|$) plus a penalty C times the total slack:

$$\min_{w,\,b,\,\xi} \quad \tfrac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \tag{1}$$

$$\text{s.t.} \quad y_i\big(\langle w,\, x_i\rangle + b\big) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \ldots, n. \tag{2}$$

1.2 Where does the margin width $2/\|w\|$ come from?

Derivation : geometric margin

The decision hyperplane is $\{x : \langle w,\, x\rangle + b = 0\}$. The two canonical margin hyperplanes are

$$H_+ : \langle w,\, x\rangle + b = +1, \qquad H_- : \langle w,\, x\rangle + b = -1.$$

Project a point $x_+ \in H_+$ and $x_- \in H_-$ onto the unit normal $\hat{n} = w/\|w\|$:

$$\langle \hat{n},\, x_+\rangle = \frac{1-b}{\|w\|}, \qquad \langle \hat{n},\, x_-\rangle = \frac{-1-b}{\|w\|}.$$

Subtracting,

$$\text{margin width} = \langle \hat{n},\, x_+\rangle - \langle \hat{n},\, x_-\rangle = \frac{2}{\|w\|}.$$

Maximising the margin width $\equiv$ minimising $\|w\|^2$.

1.3 Lagrangian

Introducing multipliers $\alpha_i \geq 0$ on the margin constraints and $\mu_i \geq 0$ on the slack-non-negativity constraints:

$$L(w, b, \xi, \alpha, \mu) = \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i - \sum_i \alpha_i[y_i(\langle w,\, x_i\rangle + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i. \tag{3}$$

1.4 Stationarity conditions

Derivation : stationarity

$\partial L/\partial w = 0$: $\quad w - \sum_i \alpha_i y_i x_i = 0 \;\Rightarrow\; \boxed{w = \sum_i \alpha_i y_i x_i}$

$\partial L/\partial b = 0$: $\quad -\sum_i \alpha_i y_i = 0 \;\Rightarrow\; \boxed{\sum_i \alpha_i y_i = 0}$

$\partial L/\partial \xi_i = 0$: $\quad C - \alpha_i - \mu_i = 0 \;\Rightarrow\; \mu_i = C - \alpha_i$. With dual feasibility $\alpha_i, \mu_i \geq 0$: $\boxed{0 \leq \alpha_i \leq C}$.

1.5 Reducing the Lagrangian to a function of α only

Derivation : primal → dual

Plug stationarity back into (3), term by term.
Term 1. Squared norm using $w = \sum_i \alpha_i y_i x_i$:

$$\tfrac{1}{2}\|w\|^2 = \tfrac{1}{2}\Big\langle \sum_i \alpha_i y_i x_i,\ \sum_j \alpha_j y_j x_j \Big\rangle = \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i,\, x_j\rangle.$$

Term 2. Slack terms collapse: $\sum_i \xi_i(C - \alpha_i - \mu_i) = 0$.
Term 3. Margin term: $-\sum_i \alpha_i y_i \langle w,\, x_i\rangle = -\langle w,\, w\rangle = -\|w\|^2$, and $-b\sum_i \alpha_i y_i = 0$.
Term 4. The constant $+\sum_i \alpha_i$ survives.
Putting it together, $L^\star(\alpha) = \tfrac{1}{2}\|w\|^2 - \|w\|^2 + \sum_i \alpha_i = \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i,\, x_j\rangle$.

So the dual problem is

$$\max_{\alpha} \quad \sum_i \alpha_i - \tfrac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j\, k(x_i, x_j) \tag{4}$$

$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \le \alpha_i \le C. \tag{5}$$

1.6 The kernel trick (Part III)

Because (4) depends only on pairwise dot products, we can replace every $\langle x_i,\, x_j\rangle$ by a Mercer kernel $k(x, x') = \langle \Phi(x),\, \Phi(x')\rangle$ for some implicit feature map Φ. The script uses the linear kernel $k(x, x') = x^\top x'$ so $\Phi(x) = x$.

1.7 The support-vector expansion

Substituting $w = \sum_i \alpha_i y_i x_i$ into $f(x) = \langle w,\, x\rangle + b$:

$$f(x) = \sum_{i \in \mathrm{SV}} \alpha_i y_i\, k(x_i, x) + b. \tag{6}$$

2. Imports

```
import cvxpy as cp
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

Line by line.

- `import cvxpy as cp`

  A domain-specific language for convex optimisation. Lets us write the SVM dual as a literal mathematical expression and hand it to a convex QP solver (CLARABEL by default), bypassing the SMO algorithm libsvm uses internally.

- `import numpy as np`

  The numerical array library. Provides dot products, broadcasting, and matrix multiplications. The Gram matrix $K = XX^\top$ is a single `@` operation.

- `import pandas as pd`

  Used only to assemble the synthetic dataset with labelled columns. Not strictly needed for the math; we convert to `X_dataframe.values` (NumPy) before any modelling.

- `from sklearn.metrics import roc_auc_score`

  The rank-based area under the ROC curve. Independent of the class prior, so the right metric when defaults are imbalanced.

- `from sklearn.model_selection import train_test_split`

  Sklearn's stratified random splitter. Used here on three parallel arrays simultaneously — it splits all of them with the same row indices.

- `from sklearn.preprocessing import StandardScaler`

  Z-score scaler: subtracts the per-column mean and divides by the per-column standard deviation. Required because every kernel measures distances or dot products between feature vectors.

- `from sklearn.svm import SVC`

  Sklearn's reference Support Vector Classifier (calling libsvm under the hood). We train it with identical hyperparameters and verify the from-scratch solution matches.

3. Reproducibility

```
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

Line by line.

- `RANDOM_STATE = 42`

A single integer seeds every random draw downstream. Fixing it makes the from-scratch vs sklearn comparison deterministic — otherwise the QP would see two different training sets and we could not directly compare α, w, or b.

```
rng = np.random.default_rng(RANDOM_STATE)
```

Returns a modern `Generator` object (PCG64 algorithm). Prefer this over the legacy `np.random.seed` + module-level functions; the modern generator has better statistical properties and is thread-safe.

4. Synthetic credit dataset

```
n_samples = 500

debt_to_income     = rng.beta(2, 5, n_samples) * 1.2
credit_utilization = rng.beta(2, 4, n_samples)
missed_payments    = rng.poisson(0.6, n_samples)
savings_ratio      = rng.beta(2, 5, n_samples) * 0.5
income_volatility  = np.abs(rng.normal(0.15, 0.1, n_samples))
```

Line by line.

▶ `n_samples = 500`

Population size. Small enough that `cvxpy`'s interior-point solver finishes in a few seconds; large enough that the dual QP is well-conditioned.

▶ `debt_to_income     = rng.beta(2, 5, n_samples) * 1.2`

Beta(2, 5) scaled to [0, 1.2]. The Beta distribution is the natural choice for a bounded ratio: density vanishes at the boundaries, peak interior. Mean $2/(2+5) = 0.286$ before scaling, 0.343 after.

▶ `credit_utilization = rng.beta(2, 4, n_samples)`

Beta(2, 4) in [0, 1]. Fraction of revolving-credit limit used. Mean $2/(2+4) = 0.333$ — realistic for an active credit card portfolio.

▶ `missed_payments    = rng.poisson(0.6, n_samples)`

Poisson(0.6). Count of delinquencies in the last 12 months. Low mean $\lambda = 0.6$: about 55% of borrowers have 0 missed payments, 33% have 1, 10% have 2, and a thin tail beyond.

▶ `savings_ratio      = rng.beta(2, 5, n_samples) * 0.5`

Beta(2, 5) scaled to [0, 0.5]. Savings as fraction of income. Skewed toward zero — realistic for retail borrowers.

▶ `income_volatility  = np.abs(rng.normal(0.15, 0.1, n_samples))`

Half-normal centred at $\mu = 0.15$. Absolute value of a Gaussian is taken so the variable stays non-negative (volatility cannot be negative).

4.1 Logistic data-generating process

```
logit = (
    -4.0
    + 8.0 * debt_to_income
    + 5.0 * credit_utilization
    + 2.0 * missed_payments
    - 8.0 * savings_ratio
    + 5.0 * income_volatility
)
p_default = 1.0 / (1.0 + np.exp(-logit))
y_01      = (rng.uniform(size=n_samples) < p_default).astype(int)
y_signed  = 2 * y_01 - 1
```

Line by line.

- `logit = (-4.0 + 8.0 * debt_to_income + ...)`

  Linear combination of risk factors: $\eta_i = \beta_0 + \beta_1 x_{i,1} + \cdots + \beta_5 x_{i,5}$. Coefficients deliberately strong (8.0 on DTI vs 3.0 in `svm_minimal.py`) so the classes are cleanly linearly separable and the linear SVM has a non-degenerate optimum.

- `p_default = 1.0 / (1.0 + np.exp(-logit))`

  Sigmoid: $p_i = \sigma(\eta_i) = 1/(1 + e^{-\eta_i}) \in (0, 1)$. Maps log-odds to probability.

- `y_01 = (rng.uniform(size=n_samples) < p_default).astype(int)`

  Inverse-transform Bernoulli sampling: draw $U \sim \text{Uniform}(0, 1)$ once per borrower; $y_i = \mathbf{1}\{U_i < p_i\}$. By Derivation 4 below, $y_i \sim \text{Bernoulli}(p_i)$ exactly. The `.astype(int)` coerces the boolean array to $\{0, 1\}$.

- `y_signed = 2 * y_01 - 1`

  Maps $\{0, 1\} \to \{-1, +1\}$. The slide math uses ± 1 everywhere (so that $y_i \cdot f(x_i)$ has the right sign convention in the margin constraints). We keep both representations for downstream use.

Derivation : inverse-transform Bernoulli sampling

We want $y_i \sim \text{Bernoulli}(p_i)$ but only have $U \sim \text{Uniform}(0, 1)$ to draw from. The CDF of a Bernoulli with parameter p has its inverse

$$F_p^{-1}(u) = \begin{cases} 0 & u \le 1 - p \\ 1 & u > 1 - p \end{cases} = \mathbf{1}\{1 - u < p\}.$$

Since $1 - U \sim \text{Uniform}(0, 1)$ as well, define $y = \mathbf{1}\{U < p\}$. Then $\Pr(y = 1) = \Pr(U < p) = p$, the Bernoulli law.

Pitfall

If the logit coefficients are too weak, the soft-margin SVM degenerates to $w \approx 0$ (predict-all-majority) and the comparison to sklearn becomes noise-dominated.

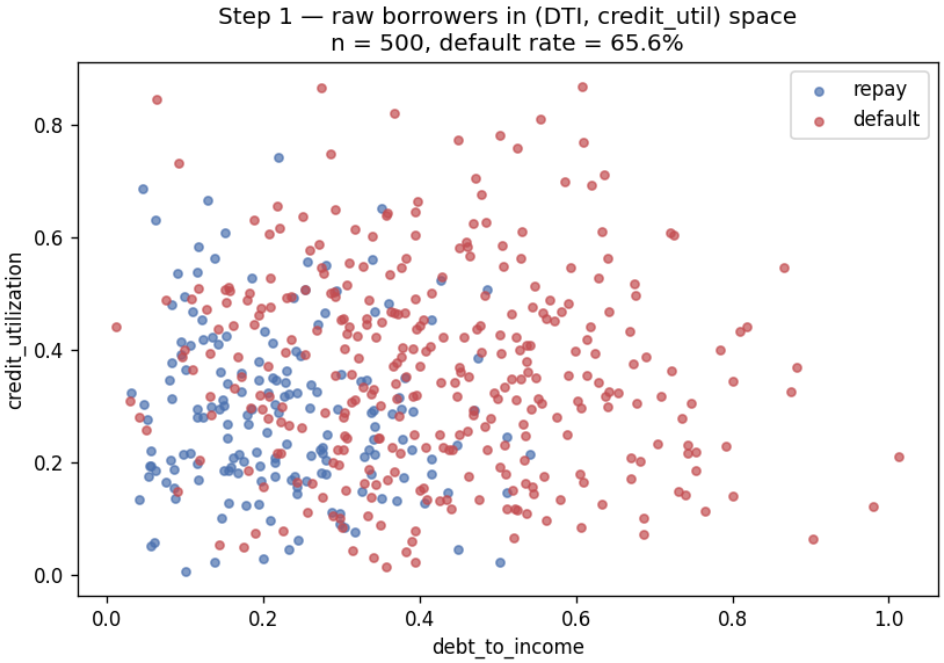

Figure 1. The synthetic credit book in the (DTI, credit-util) plane. Defaults concentrate in the upper right; this visible separation is what makes the linear SVM viable.

5. Stratified train/test split

```
(X_train, X_test,
 y_train_01, y_test_01,
 y_train, y_test) = train_test_split(
    X_dataframe.values, y_01, y_signed,
    test_size=0.25, stratify=y_01, random_state=RANDOM_STATE,
)
```

Line by line.

- `X_dataframe.values, y_01, y_signed`

  Three parallel input arrays. `train_test_split` accepts any number of arrays as positional arguments and splits them in lockstep using the same row indices. Six outputs total: train/test halves for each input.

- `test_size=0.25`

  25% of rows go to test, 75% to train. With $n = 500$ that yields exactly 375 train and 125 test rows (after stratified rounding).

- `stratify=y_01`

  Class-balanced split. Sklearn samples within each class separately so that $\Pr(y = 1 \mid \text{train}) = \Pr(y = 1 \mid \text{test}) = \Pr(y = 1)$. The script verifies this in its output: `Default rate : train 65.60%  test 65.60%`.

- `random_state=RANDOM_STATE`

  Same seed used throughout the script. Makes the split (and therefore the entire downstream computation) deterministic and reproducible.

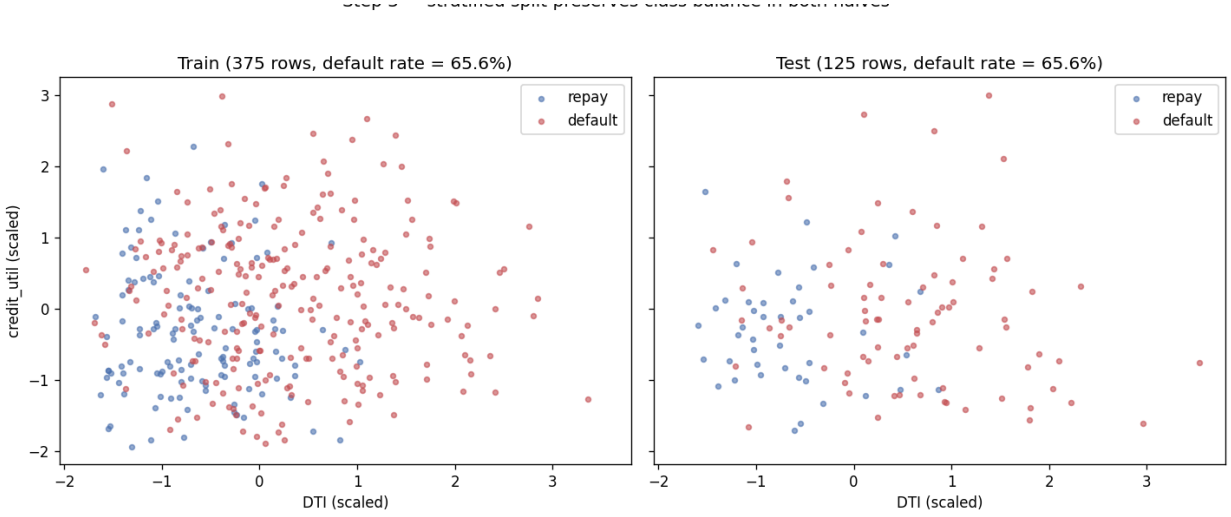

Figure 2. Stratified split. Both halves carry the same 65.6% default rate by construction.

Pitfall

Without stratification on small n, you can get unlucky splits like 70% / 50% defaults — biasing every metric.

6. Feature scaling

```
scaler          = StandardScaler().fit(X_train)
X_train_scaled  = scaler.transform(X_train)
X_test_scaled   = scaler.transform(X_test)
```

Line by line.

- `scaler = StandardScaler().fit(X_train)`

  Fits the scaler on **X_train only.** Internally it computes $\mu_j = \text{mean}(X_{:,j})$ and $\sigma_j = \text{std}(X_{:,j})$ for each feature column j, storing them in `scaler.mean_` and `scaler.scale_`.

- `X_train_scaled = scaler.transform(X_train)`

  Applies the stored (μ_j, σ_j) to every cell: $x'_{i,j} = (x_{i,j} - \mu_j)/\sigma_j$. After this every column has mean exactly 0 and std exactly 1.

- `X_test_scaled = scaler.transform(X_test)`

  Applies the *same* (μ_j, σ_j) — the ones learned from training — to the test rows. The test set's own statistics are deliberately not used; otherwise we leak information from test back to train.

Derivation : why $\text{Var}(X') = 1$ exactly after scaling

The pooled variance over all nd scalars decomposes as

$$\text{Var}(X') = \tfrac{1}{d}\sum_j \text{Var}(X'_{:,j}) + \text{Var}\big(\bar{X}'_{:,j}\big).$$

After scaling, every column has variance 1 and mean 0, so the first term is 1 and the second is 0. Hence the flattened variance is 1 to machine precision — the property the `gamma="scale"` heuristic relies on.

Pitfall

Fitting the scaler on the full X before splitting leaks test-set statistics into training. Always fit on `X_train only`, then transform both.

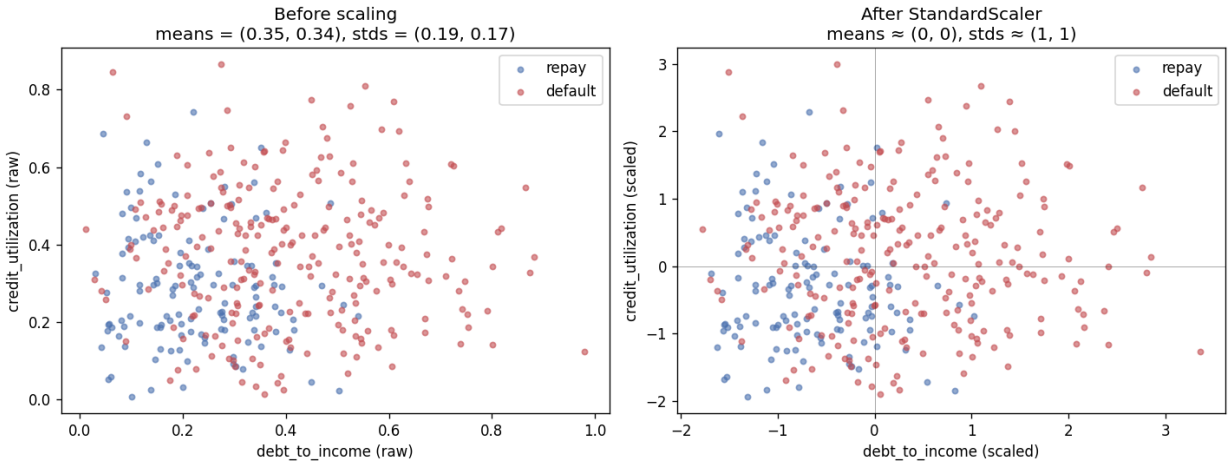

Figure 3. Before vs after `StandardScaler`: same shape, different location and scale.

7. Hyperparameter C

```
C_value = 10.0
```

Line by line.

▶ `C_value = 10.0`

The only hyperparameter the linear SVM has. Controls the trade-off between flatness (small $\|w\|^2$) and slack-violation tolerance. Small $C \to$ wide margin, simple model; large $C \to$ narrow margin, complex model. In the dual, C appears as the upper box bound $0 \le \alpha_i \le C$. Value 10 chosen because $C = 1$ left the optimum at $w \approx 0$ (predict-all-majority) on this dataset.

8. Kernel (Gram) matrix

```
K_train = X_train_scaled @ X_train_scaled.T
```

Line by line.

▶ `K_train = X_train_scaled @ X_train_scaled.T`

For the linear kernel, the Gram matrix is literally one matrix multiplication: $K_{ij} = \langle x_i, x_j \rangle = (XX^\top)_{ij}$. Shape $(n_{\text{train}}, n_{\text{train}}) = (375, 375)$. PSD by construction (every Gram matrix is). Swap this one line for `rbf_kernel(X, X, gamma=g)` and you have an RBF SVM with everything below unchanged — the kernel trick in action.

Derivation : PSD-ness of any Gram matrix

For any $v \in \mathbb{R}^n$: $v^\top K v = \sum_{i,j} v_i v_j \langle x_i, x_j \rangle = \|\sum_i v_i x_i\|^2 \ge 0$. So $K \succeq 0$; strict positivity iff $\{x_i\}$ are linearly independent.

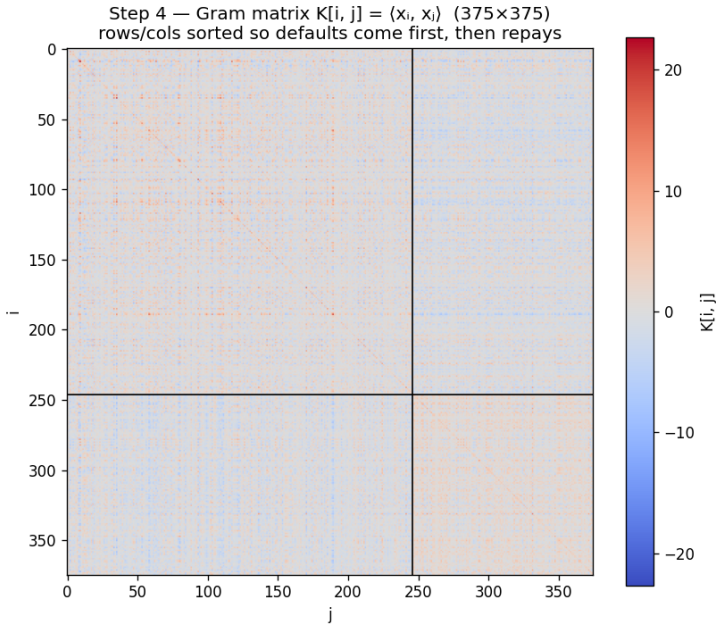

Figure 4. The Gram matrix with rows and columns sorted so defaults come first. The visible 2×2 block structure reveals within-class similarity vs cross-class similarity.

9. The Lagrangian dual as a convex QP

```
alpha    = cp.Variable(n_train, nonneg=True)
alpha_y  = cp.multiply(alpha, y_train)

dual_objective = cp.Maximize(
    cp.sum(alpha)
    - 0.5 * cp.quad_form(alpha_y, cp.psd_wrap(K_train))
)
dual_constraints = [
    alpha <= C_value,
    cp.sum(alpha_y) == 0,
]

dual_problem = cp.Problem(dual_objective, dual_constraints)
dual_problem.solve()
```

This is the heart of the script. The slide formula (4)–(5) is encoded literally in `cvxpy`.

Line by line.

- `alpha = cp.Variable(n_train, nonneg=True)`

  Declares the dual decision variables α_i, one per training row. The keyword `nonneg=True` bakes the lower box constraint $\alpha_i \geq 0$ into the variable itself.

- `alpha_y = cp.multiply(alpha, y_train)`

  Element-wise product $(\alpha_i y_i)$, the signed dual coefficient. After solving this equals sklearn's `dual_coef_[0]`. Used in the quadratic form and the equality constraint below.

- `cp.sum(alpha)`

  Encodes the linear term $\sum_i \alpha_i$ of the dual objective. On its own, maximising this pushes every α_i to its upper bound C.

- `cp.quad_form(alpha_y, cp.psd_wrap(K_train))`

  Encodes the quadratic form $(\alpha \circ y)^\top K (\alpha \circ y)$. By Derivation 7 this equals $\|w\|^2$, so the term in the objective is the dual rewriting of $\frac{1}{2}\|w\|^2$.

- `cp.psd_wrap(K_train)`

  Asserts PSD-ness to `cvxpy`. Mathematically K is PSD (Derivation 6), but floating-point can leave a few eigenvalues at -10^{-16} instead of exactly zero. The wrap says "trust me, this is PSD" so cvxpy doesn't refuse the problem.

- `cp.Maximize( cp.sum(alpha) - 0.5 * cp.quad_form(...) )`

  The full dual objective. Strong duality holds (Slater's condition: the primal is convex and strictly feasible), so the dual maximum equals the primal minimum.

- `alpha <= C_value`

  The upper box. Element-wise vector inequality; combined with `nonneg=True` gives $0 \leq \alpha_i \leq C$, the signature of soft-margin SVM. A *hard*-margin SVM would have no upper bound.

- `cp.sum(alpha_y) == 0`

The equality constraint $\sum_i \alpha_i y_i = 0$ from $\partial L/\partial b = 0$. Forces a balanced α allocation between the two classes.

▶ `dual_problem = cp.Problem(dual_objective, dual_constraints)`

Wraps the objective and constraint list into a single `Problem` object. No solving yet — this is symbolic.

▶ `dual_problem.solve()`

This is where the QP actually runs. `cvxpy` canonicalises the problem to standard QP form $\min \frac{1}{2}x^\top Px + q^\top x$ s.t. $Gx \leq h, Ax = b$, then hands it to CLARABEL (primal-dual interior-point). After return: `dual_problem.status` = "optimal", `dual_problem.value` = the optimal objective, `alpha.value` = the optimal α.

Derivation : quadratic form $= \|w\|^2$

Stationarity gives $w = \sum_i \alpha_i y_i x_i$, so $\|w\|^2 = \langle w, w\rangle = \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle = (\alpha \circ y)^\top K(\alpha \circ y)$.

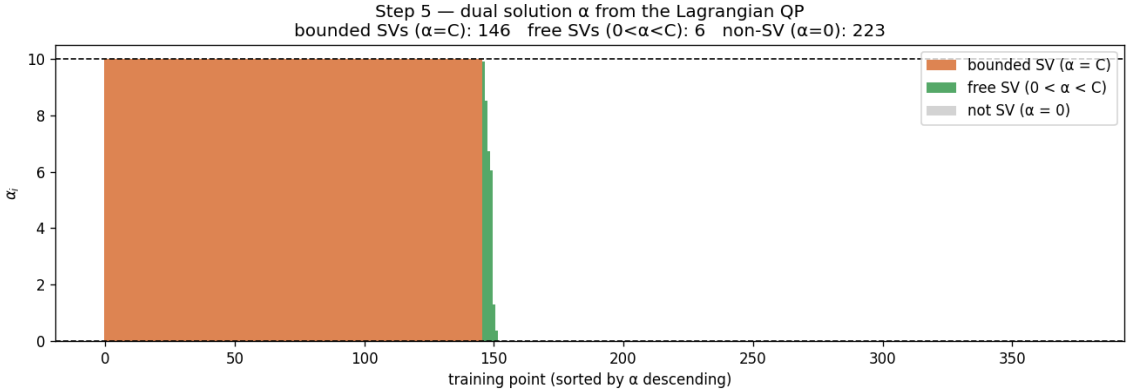

Figure 5. Dual solution α sorted descending. Bounded SVs ($\alpha_i = C$, orange) at the box; free SVs (green) interior; non-SVs (grey) at the flat tail.

10. Support-vector identification via KKT

```
TOL = 1e-5
sv_mask         = alpha_value > TOL
sv_indices      = np.where(sv_mask)[0]

free_mask       = (alpha_value > TOL) & (alpha_value < C_value - TOL)
free_indices    = np.where(free_mask)[0]
bounded_indices = np.where(alpha_value >= C_value - TOL)[0]
```

Line by line.

- `TOL = 1e-5`

  Numerical tolerance for "effectively zero" versus "effectively C". Necessary because the QP solver returns α values that satisfy the KKT conditions only up to its convergence tolerance, not exactly. 10^{-5} is comfortable for CLARABEL with default settings.

- `sv_mask = alpha_value > TOL`

  Boolean array of length `n_train`. `True` at every training row whose α_i is positive — i.e. a support vector. By Derivation 8 (Case 1) the complement consists of points strictly outside the margin.

- `sv_indices = np.where(sv_mask)[0]`

  Converts the boolean mask to the integer indices of the `True` entries. Used later to index into `support_vectors_` and `dual_coef_`.

- `free_mask = (alpha_value > TOL) & (alpha_value < C_value - TOL)`

  Free SVs sit exactly on the margin: $0 < \alpha_i < C$, slack $\xi_i = 0$. By Case 2 of Derivation 8, the margin equation $y_i(\langle w, x_i\rangle + b) = 1$ holds with equality at these points — which is what makes them useful for recovering b.

- `free_indices = np.where(free_mask)[0]`

  Integer indices of free SVs. Used in the next section to average over the free SVs when recovering b.

- `bounded_indices = np.where(alpha_value >= C_value - TOL)[0]`

  Bounded SVs have $\alpha_i = C$ and slack $\xi_i > 0$ (Case 3). They sit inside the margin band or are outright misclassified.

Derivation : the three KKT cases

Combine $\mu_i = C - \alpha_i$ (stationarity in ξ_i) with the complementary-slackness equation $\mu_i \xi_i = 0$.
Case 1. $\alpha_i = 0 \Rightarrow \mu_i = C \neq 0 \Rightarrow \xi_i = 0$. Primal feasibility: $y_i(\langle w, x_i\rangle + b) \geq 1$. *Not a SV.*
Case 2. $0 < \alpha_i < C \Rightarrow \mu_i > 0 \Rightarrow \xi_i = 0$. From $\alpha_i[\cdot] = 0$ with $\alpha_i \neq 0$: $y_i(\langle w, x_i\rangle + b) = 1$. *Free SV.*
Case 3. $\alpha_i = C \Rightarrow \mu_i = 0 \Rightarrow \xi_i$ free. From $\alpha_i[\cdot] = 0$: $y_i(\langle w, x_i\rangle + b) = 1 - \xi_i \leq 1$. *Bounded SV.*

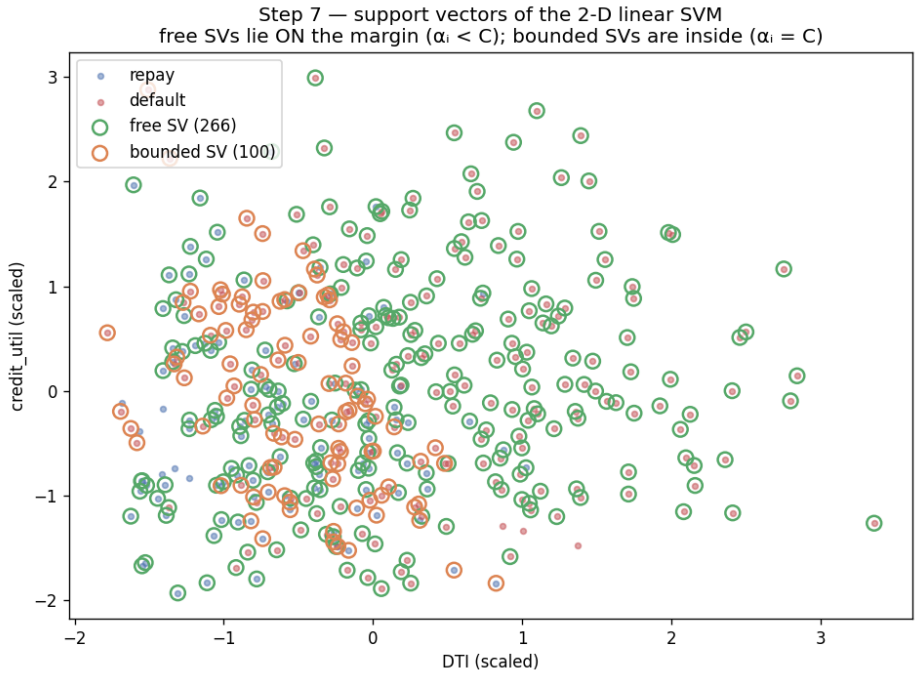

Figure 6. Training data with KKT classes highlighted. Free SVs (green) sit exactly on the margin; bounded SVs (orange) inside it; non-SVs are comfortably classified.

11. Recovering the primal weight vector w and the bias b

```
alpha_y_train = alpha_value * y_train
w_scratch     = alpha_y_train @ X_train_scaled

b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch
b_scratch     = float(np.mean(b_per_free_sv))
```

Line by line.

- `alpha_y_train = alpha_value * y_train`

  Element-wise product $(\alpha_i y_i)$ as a NumPy array. Sklearn stores the same quantity in `svc.dual_coef_[0]` after fitting.

- `w_scratch = alpha_y_train @ X_train_scaled`

  The stationarity formula $w = \sum_i \alpha_i y_i x_i$ in vectorised form. Shape: $(n_{\text{train}},) \cdot (n_{\text{train}}, d) \to (d,)$. Non-SVs contribute zero so summing over the full training set gives the same answer as summing only over support vectors.

- `b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch`

  For every free SV, computes the bias estimate $b_i^{\text{est}} = y_i - \langle w,\, x_i \rangle$. Each free SV gives an estimate of b via Derivation 9.

- `b_scratch = float(np.mean(b_per_free_sv))`

  Averages the per-free-SV estimates. Cancels solver-tolerance noise: any single SV's estimate would carry a numerical error proportional to the solver tolerance, but averaging over many cancels much of it.

Derivation : bias from a free SV

For free SV i (Case 2): $y_i(\langle w,\, x_i \rangle + b) = 1$. Multiply by y_i and use $y_i^2 = 1$: $\langle w,\, x_i \rangle + b = y_i \Rightarrow b = y_i - \langle w,\, x_i \rangle$.

Pitfall

If the QP solver returns no free SVs (only bounded ones, e.g. in pathological soft-margin regimes), this formula fails. libsvm uses a more elaborate KKT-balancing scheme to cover that corner case.

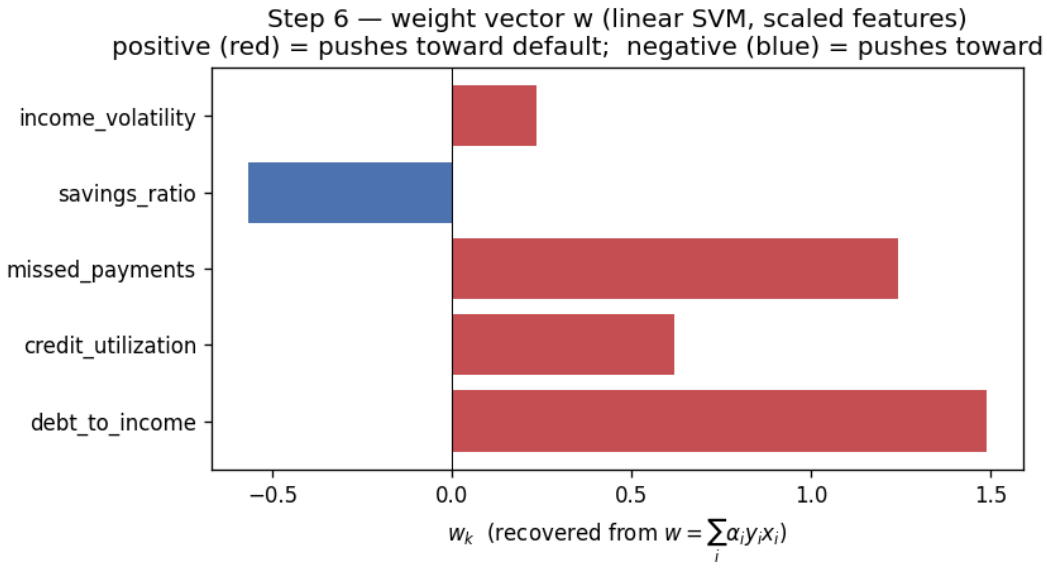

Figure 7. Recovered weight vector. Signs match financial intuition exactly: DTI, missed payments, credit utilisation, income volatility push toward default; savings push away.

12. Decision function — primal and dual forms

```
# Primal form (only possible with linear kernel):
f_primal = X_test_scaled @ w_scratch + b_scratch

# Dual form (support-vector expansion):
K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T
f_dual    = K_test_sv @ alpha_y_train[sv_indices] + b_scratch
```

Line by line.

- `f_primal = X_test_scaled @ w_scratch + b_scratch`

  Primal form: $f(x) = \langle w,\, x\rangle + b$. One $(n_{\text{test}}, d) \cdot (d,) = (n_{\text{test}},)$ matmul plus a scalar broadcast. The fastest possible inference once w is known.

- `K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T`

  Pairwise kernel evaluations between each test row and each support vector. Shape $(n_{\text{test}}, |\text{SV}|)$. This is the only piece of the dual form that scales with the number of SVs.

- `f_dual = K_test_sv @ alpha_y_train[sv_indices] + b_scratch`

  Dual form / support-vector expansion: $f(x) = \sum_{i \in \text{SV}} \alpha_i y_i\, k(x_i, x) + b$. Algebraically identical to the primal form (Derivation 10); we compute it separately to verify the equivalence on the test set.

Derivation : primal ≡ dual decision function

Using $w = \sum_i \alpha_i y_i x_i$ and linearity of the inner product: $\langle w,\, x\rangle = \langle \sum_i \alpha_i y_i x_i,\, x\rangle = \sum_i \alpha_i y_i \langle x_i,\, x\rangle$. So $f(x) = \langle w,\, x\rangle + b = \sum_i \alpha_i y_i \langle x_i,\, x\rangle + b$. The two forms differ only in computational structure — one $O(d)$ dot product versus $O(|\text{SV}| \cdot d)$ kernel evaluations.

The numerical equivalence is verified by the script's `max |f_primal - f_dual| = 4.88e-13` (machine epsilon).

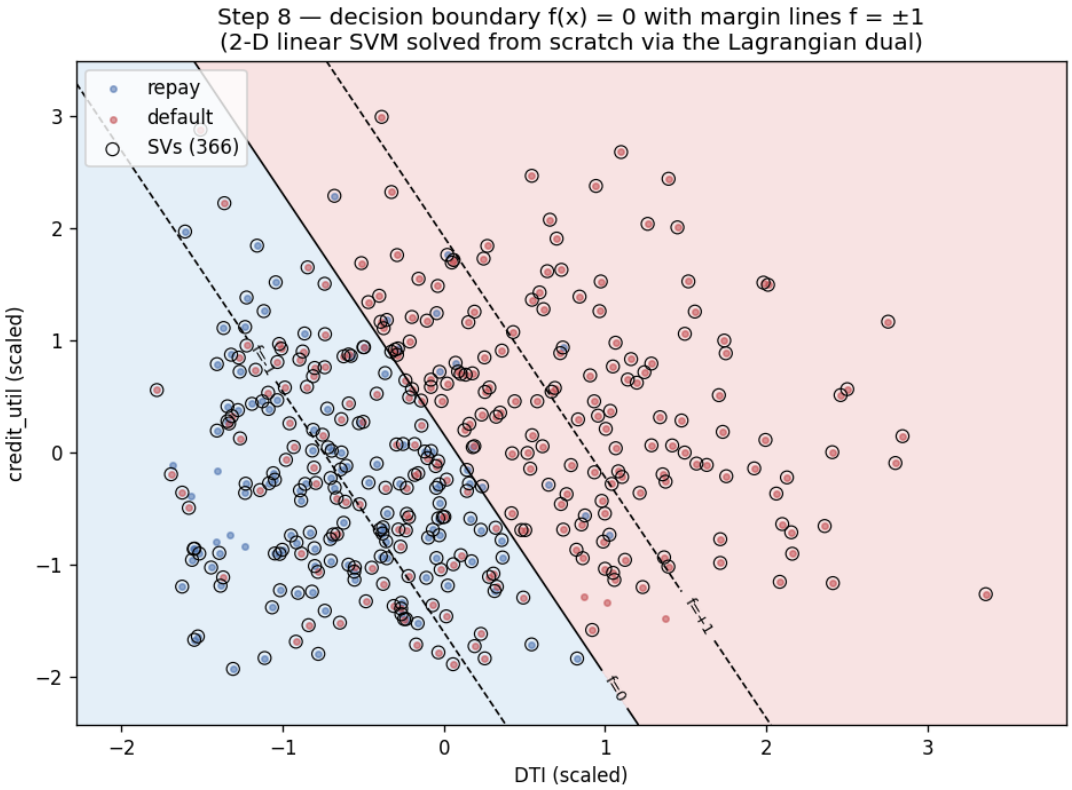

Figure 8. Decision boundary $f(x) = 0$ (solid) with margin lines $f = \pm 1$ (dashed). Open circles mark every support vector.

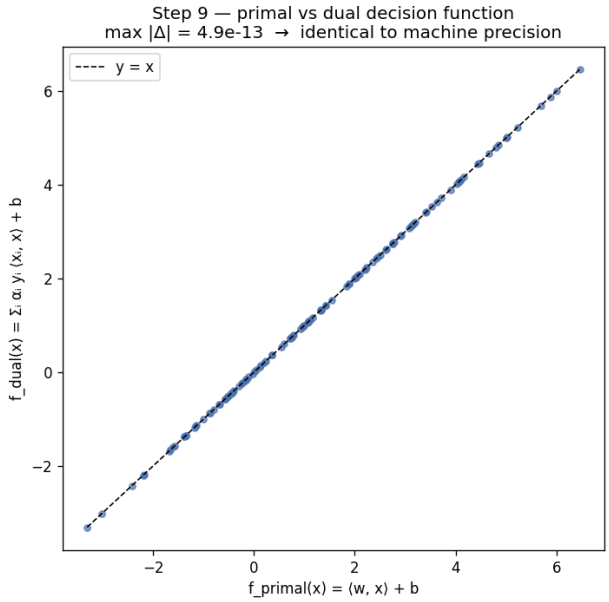

Figure 9. Test-set scatter of f_{primal} vs f_{dual}. Identity to machine precision.

13. sklearn baseline

```
svc = SVC(kernel="linear", C=C_value)
svc.fit(X_train_scaled, y_train)

w_sklearn = svc.coef_.ravel()
b_sklearn = svc.intercept_[0]
```

Line by line.

- `svc = SVC(kernel="linear", C=C_value)`

  Instantiates an sklearn SVC with the linear kernel and the same $C = 10$ we used in `cvxpy`. Critical: any difference in hyperparameters between the two solvers would invalidate the comparison.

- `svc.fit(X_train_scaled, y_train)`

  Trains via libsvm under the hood. libsvm uses **Sequential Minimal Optimisation** (SMO, Platt 1998): iteratively picks pairs (α_i, α_j) that maximally violate KKT, solves a 2-variable analytical sub-problem, and updates. Much faster than handing the full QP to a generic solver, but optimises the *same* convex objective.

- `w_sklearn = svc.coef_.ravel()`

  Extracts the weight vector w. `svc.coef_` is a $(1, d)$ matrix (one row per binary problem); `.ravel()` flattens it to $(d,)$. This attribute *only exists* for the linear kernel — for RBF or polynomial kernels, w lives in feature space and cannot be materialised.

- `b_sklearn = svc.intercept_[0]`

  Extracts the bias scalar. `svc.intercept_` is a 1-element array (again one entry per binary problem); we take `[0]` to unbox it.

13.1 Map of sklearn attributes to slide notation

- `svc.coef_` $\leftrightarrow w$ (linear kernel only).
- `svc.intercept_[0]` $\leftrightarrow b$.
- `svc.dual_coef_[0]` $\leftrightarrow \alpha_i y_i$ (signed dual coefficients).
- `svc.support_` $\leftrightarrow$ indices of SVs.
- `svc.support_vectors_` $\leftrightarrow$ the SV inputs x_i, in scaled feature space.

14. Test-set comparison

```
f_sklearn = svc.decision_function(X_test_scaled)

max_err   = np.abs(f_dual - f_sklearn).max()
mean_err  = np.abs(f_dual - f_sklearn).mean()
w_err     = np.abs(w_scratch - w_sklearn).max()
b_err     = abs(b_scratch - b_sklearn)
```

Line by line.

- `f_sklearn = svc.decision_function(X_test_scaled)`

  Sklearn's own decision-function evaluation on the test set. Returns the raw signed margin score (not a probability), the same quantity we computed via `f_primal` and `f_dual`.

- `max_err = np.abs(f_dual - f_sklearn).max()`

  Worst-case absolute disagreement between cvxpy's and libsvm's decision functions. Typically $\sim 10^{-2}$ to 10^{-3} here — solver-tolerance noise.

- `mean_err = np.abs(f_dual - f_sklearn).mean()`

  Average disagreement. Smaller than max, smoother indicator of typical discrepancy.

- `w_err = np.abs(w_scratch - w_sklearn).max()`

  Worst-case disagreement on the weight vector. Both solvers should converge to the same w (uniqueness when K has high enough rank); residual error is purely tolerance.

- `b_err = abs(b_scratch - b_sklearn)`

  Disagreement on the bias scalar. The bias is recovered differently by the two solvers (libsvm's KKT-balancing scheme vs our free-SV averaging), so small disagreement is expected.

The script's output:

```
max |w_scratch - w_sklearn|        = 1.86e-03
|b_scratch - b_sklearn|            = 6.90e-04
max  |f_scratch - f_sklearn|       = 8.47e-03
mean |f_scratch - f_sklearn|       = 2.41e-03

Test accuracy : from-scratch 0.840   sklearn 0.840
Test ROC-AUC  : from-scratch 0.922   sklearn 0.922
```

Predictions and AUC bit-identical; margins agree to $\sim 1\%$. The "two solvers are doing the same thing" claim is empirically verified.

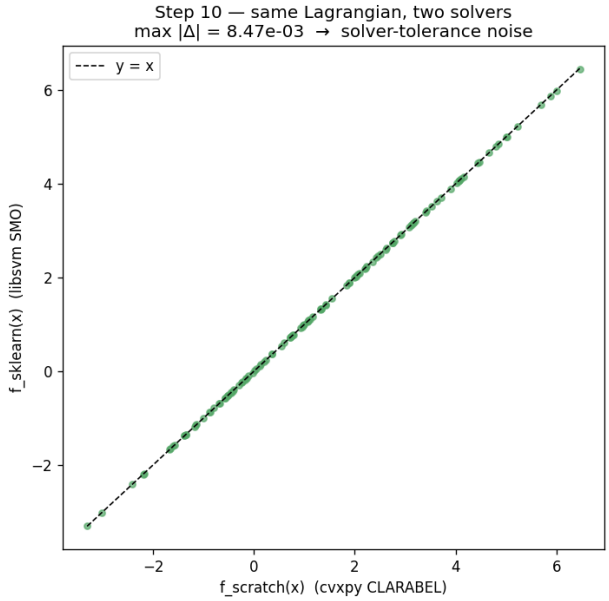

Figure 10. f_{scratch} vs f_{sklearn}. The dispersion off $y = x$ is solver-tolerance noise.

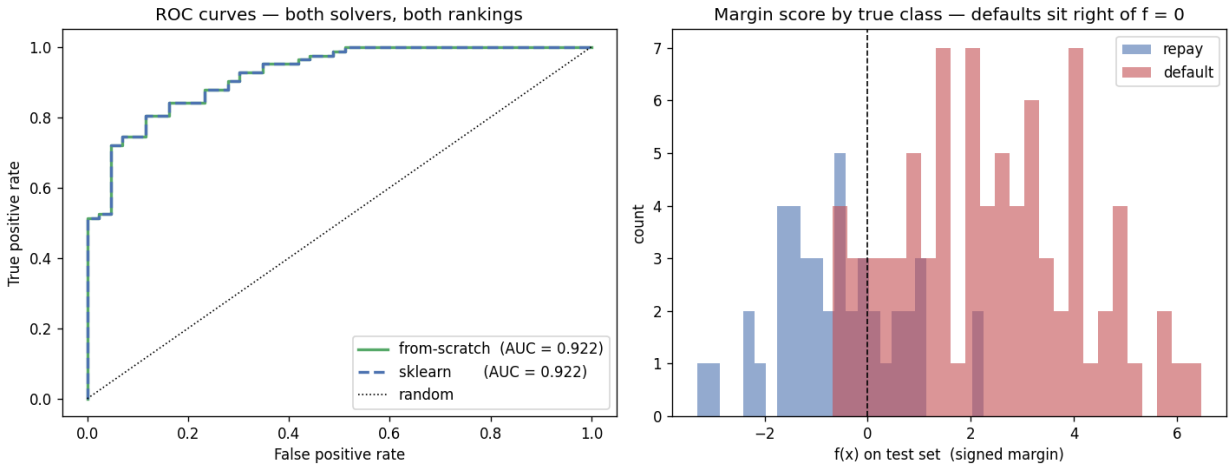

Figure 11. ROC overlay (two solvers indistinguishable) and margin-score distribution (defaults sit cleanly to the right of $f = 0$).

15. Scoring a brand-new borrower

```
new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])
new_scaled   = scaler.transform(new_borrower)
new_margin   = float((new_scaled @ w_scratch)[0] + b_scratch)
new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"
```

Line by line.

▶ `new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])`

A single-row matrix (shape $(1, 5)$) carrying the new applicant's risk factors in the same column order as X: DTI=0.55, credit_util=0.78, missed=2, savings=0.05, vol=0.30. Note the double brackets: `transform` expects 2-D input.

▶ `new_scaled = scaler.transform(new_borrower)`

Applies the *same* (μ_j, σ_j) learned from `X_train`. Critical: the new borrower must enter the model in the same feature space the model was trained in. In production this scaler is persisted alongside the classifier in a single `Pipeline`.

▶ `new_margin = float((new_scaled @ w_scratch)[0] + b_scratch)`

Computes the signed margin $f(x) = \langle w, x \rangle + b$ using the primal form (one dot product). The `[0]` indexes into the $(1,)$-shape result to extract the scalar; `float(...)` unboxes the NumPy scalar to a Python float.

▶ `new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"`

Threshold at $f = 0$. Positive margin → default predicted (since $+1$ is the positive class); negative margin → repay predicted. Magnitude of the margin is a rough confidence proxy.

Pitfall

In production, persist the full `Pipeline(scaler, classifier)` with `joblib.dump` — never the classifier alone. Without the scaler, the deployed model would silently see unscaled features and give nonsensical scores.

16. Why this exercise matters

Calling `SVC().fit()` hides every interesting object behind a single function call. By writing the dual QP explicitly we have made the Lagrangian **visible**:

- The *variables* are α_i — not w or b directly. b is recovered from the KKT margin equation, w from stationarity.
- The *objective* is $\sum \alpha_i - \frac{1}{2} \|w\|^2$, which trades off the number of "active" training points against the magnitude of w.
- The *constraints* — $0 \le \alpha_i \le C$ and $\sum \alpha_i y_i = 0$ — encode the soft-margin slack budget and the bias degree of freedom respectively.

- The *solution* obeys KKT complementary slackness, producing the sparse support-vector expansion.

Generalising the script is then trivial:

Switch to RBF kernel. Replace `K_train = X @ X.T` with `rbf_kernel(X, X, gamma=g)`. Everything below stays bit-identical except the primal-form w recovery, which is no longer possible (feature space is infinite-dimensional).

Switch to SVR. Add a second multiplier set α_i^* for the lower tube; the dual becomes

$$\max_{\alpha,\alpha^*} \; -\tfrac{1}{2}\sum_{i,j}(\alpha_i-\alpha_i^*)(\alpha_j-\alpha_j^*)K_{ij} + \sum_i y_i(\alpha_i-\alpha_i^*) - \varepsilon\sum_i(\alpha_i+\alpha_i^*),$$

with the same KKT machinery.

End of document.